

Voxelized GICP for Fast and Accurate 3D Point Cloud Registration

Kenji Koide¹, Masashi Yokozuka¹, Shuji Oishi¹, and Atsuhiko Banno¹

Abstract—This paper proposes the voxelized generalized iterative closest point (VGICP) algorithm for fast and accurate three-dimensional point cloud registration. The proposed approach extends the generalized iterative closest point (GICP) approach with voxelization to avoid costly nearest neighbor search while retaining its accuracy. In contrast to the normal distributions transform (NDT), which calculates voxel distributions from point positions, we estimate voxel distributions by aggregating the distribution of each point in the voxel. The voxelization approach allows us to efficiently process the optimization in parallel, and the proposed algorithm can run at 30 Hz on a CPU and 120 Hz on a GPU. Through evaluations in simulated and real environments, we confirmed that the accuracy of the proposed algorithm is comparable to GICP, but is substantially faster than existing methods. This will enable the development of real-time 3D LIDAR applications that require extremely fast evaluations of the relative poses between LIDAR frames.

I. INTRODUCTION

Registration of three-dimensional (3D) point clouds is a crucial task for many 3D LIDAR applications, such as calibration, localization, mapping, and environment recognition. There are two popular point cloud registration methods for 3D LIDARs: Generalized Iterative Closest Point (GICP) [1] and the Normal Distributions Transform (NDT) [2], [3]. Whereas GICP extends the classical ICP algorithm [4] in a distribution-to-distribution comparison manner for accurate registration, the NDT takes advantage of the voxelization approach to avoid costly nearest neighbor search and improve processing speed. However, both methods have their own weaknesses. Because GICP and other ICP-variants highly depend on nearest neighbor search, it is sometimes hard to run them in realtime on a computer with limited computational power if the number of points is large. In contrast, the NDT is typically very sensitive to the choice of the voxel resolution. The best voxel resolution depends on the environment and sensor properties, and if we do not choose an appropriate resolution, the registration accuracy of the NDT drastically drops.

In this paper, we propose the Voxelized GICP (VGICP) algorithm for fast and accurate 3D point cloud registration. The voxelization approach enables the proposed algorithm to efficiently run in parallel, and our VGICP implementation can process a point cloud containing 15,000 points at 30 Hz on a CPU and 120 Hz on a GPU. By aggregating the distribution of all points in a voxel (multi-point distributions to a single-voxel distribution), we estimate the distributions of

the voxels robustly. In contrast to the NDT, which estimates voxel distributions from point positions, this approach yields valid voxel distributions even when there are few points in a voxel, yielding an algorithm that is robust to changes in voxel resolution. The evaluation results obtained in simulated and real environments illustrate that the proposed VGICP algorithm shows registration accuracy that is comparable to that of GICP and outperforms other methods in terms of processing speed.

The contribution of this paper is three-fold. First, we propose a multi-point distribution aggregation approach to robustly estimate the distribution of a voxel from a smaller number of points. Second, we propose the VGICP algorithm, which is as accurate as GICP but substantially faster than existing methods. Third, the implementations are available from a public repository¹. The repository contains the implementation of the proposed VGICP as well as a parallelized implementation of GICP.

II. RELATED WORK

A. GICP

There are many variants of the classic ICP algorithm, such as Trimmed ICP [5] and Normal ICP [6]. GICP [1] is one of the most popular ICP variants. GICP extends the classical ICP algorithm in a distribution-to-distribution matching fashion. Although it is known for its good accuracy, this algorithm (and other ICP-variants) highly depend on nearest neighbor search to associate the closest points. Although an efficient KD-tree-based search is typically used, the nearest neighbor search often becomes a bottleneck so that the algorithm cannot run in realtime when the number of points is large. Furthermore, a method based on a nearest neighbor search is not suitable for optimization on a GPU because it makes heavy use of conditional branches, which decreases the performance of the GPU significantly.

B. NDT

The NDT [2] takes a voxel-based association approach instead of an exact nearest neighbor search. This algorithm first splits an input point cloud into a set of voxels and fits a normal distribution to the points in each voxel. Then, it aligns another point cloud to the voxelized one by finding the transformation that maximizes the likelihood of the input points under the distributions of the voxels. Because the NDT avoids the costly nearest neighbor association, it is typically much faster than ICP-variant algorithms. D2D-NDT (Distribution-to-Distribution NDT) [7] is an extension of

¹ All the authors are with the Department of Information Technology and Human Factors, the National Institute of Advanced Industrial Science and Technology, Umezono 1-1-1, Tsukuba, 3050061, Ibaraki, Japan, k.koide@aist.go.jp

¹https://github.com/koide3/fast_gicp

the NDT that voxelizes both the source and target point clouds and calculate the distance between the distributions of the source and target voxels. Its comparison scheme is similar to that of GICP, and [8] suggests that D2D-NDT is superior to the classic NDT in terms of accuracy. However, the accuracy of the NDT and its variants depends on the choice of the voxel size. To obtain the best performance with the NDT, we need to carefully choose an appropriate voxel size depending on the sensor and environment properties. Some studies have proposed methods to make the NDT robust to hyper-parameter changes (e.g., multi-resolution [9] and trilinear voxel smoothing [3]). However, those extensions have a negative influence on the processing speed of the NDT.

C. Feature-based registration

Feature-based registration methods first extract a number of representative features from input point clouds and then estimate the transformation from the feature correspondences. Many features have been proposed for point cloud registration, such as basic plane and edge features [10], [4], Fast Point Feature Histogram (FPFH) [11], and Signature of Histograms of Orientations (SHOT) [12]. Because these features enable correspondences between point clouds to be robustly found, feature-based methods are typically robust to initial pose errors (some do not even require an initial guess). However, because feature-based methods use only a limited number of features (which is usually much smaller than the number of input points), their accuracy is worse than that of point-based methods. Therefore, in a typical use case, a point-based fine registration is performed after a feature-based method. Feature-based and point-based registration methods are orthogonal, and they should be used to complement each other.

III. PROPOSED METHOD

In this section, we first explain the GICP algorithm and then extend it in a one-to-multiple distribution correspondence fashion to derive our VGICP algorithm.

A. GICP algorithm

We consider the estimation of the transformation $\mathbf{T}$, which aligns a set of points $\mathcal{A} = \{a_0, \dots, a_N\}$ (source point cloud) with respect to another set of points $\mathcal{B} = \{b_0, \dots, b_N\}$ (target point cloud). Following the classic ICP algorithm, we assume that the correspondences between $\mathcal{A}$ and $\mathcal{B}$ are given by nearest neighbor search: $b_i = \mathbf{T}a_i$. The GICP algorithm [1] models the surface from which a point was sampled as a Gaussian distribution: $a_i \sim \mathcal{N}(\hat{a}_i, C_i^A)$, $b_i \sim \mathcal{N}(\hat{b}_i, C_i^B)$. Then, we define the transformation error as follows:

$$\hat{d}_i = \hat{b}_i - \mathbf{T}\hat{a}_i. \quad (1)$$

The distribution of d_i is given by the reproductive property of the Gaussian distribution as

$$d_i \sim \mathcal{N}(\hat{b}_i - \mathbf{T}\hat{a}_i, C_i^B + \mathbf{T}C_i^A\mathbf{T}^T), \quad (2)$$

$$= \mathcal{N}(0, C_i^B + \mathbf{T}C_i^A\mathbf{T}^T). \quad (3)$$

The GICP algorithm finds the transformation $\mathbf{T}$ that maximizes the log likelihood of Eq. (3) as follows:

$$\mathbf{T} = \arg \max_{\mathbf{T}} \sum_i \log(p(d_i)), \quad (4)$$

$$= \arg \min_{\mathbf{T}} \sum_i d_i^T (C_i^B + \mathbf{T}C_i^A\mathbf{T}^T)^{-1} d_i. \quad (5)$$

The covariance matrix of each point is typically estimated from its k neighbors (e.g., $k = 20$). Following the suggestion in [1], each covariance matrix is regularized by replacing its eigenvalue with $(1, 1, \epsilon)$. This regularization makes GICP work as a plane-to-plane ICP.

B. Voxelized GICP algorithm

To derive the voxelized GICP algorithm, we first extend Eq. (1) so that it calculates the distances between a_i and its neighbor points $\{b_j | \|a_i - b_j\| < r\}$ as follows:

$$\hat{d}_i' = \sum_j (\hat{b}_j - \mathbf{T}\hat{a}_i). \quad (6)$$

This equation can be interpreted as smoothing the target point distributions. Then, similar to Eq. (3), the distribution of d_i' is given by

$$\hat{d}_i' \sim (\mu^{d_i'}, C^{d_i'}), \quad (7)$$

$$\mu^{d_i'} = \sum_j (\hat{b}_j - \mathbf{T}\hat{a}_i) = 0, \quad (8)$$

$$C^{d_i'} = \sum_j (C_j^B + \mathbf{T}C_i^A\mathbf{T}^T). \quad (9)$$

We estimate the transformation $\mathbf{T}$ that maximizes the log likelihood of Eq. (7) as follows:

$$\mathbf{T} = \arg \min_{\mathbf{T}} \sum_i \left(\tilde{d}_i^T \tilde{C}_i^{-1} \tilde{d}_i \right), \quad (10)$$

$$\tilde{d}_i = \sum_j (b_j - \mathbf{T}a_i), \quad (11)$$

$$\tilde{C}_i = \sum_j (C_j^B + \mathbf{T}C_i^A\mathbf{T}^T). \quad (12)$$

To efficiently calculate the above equation, we modify it to

$$\mathbf{T} = \arg \min_{\mathbf{T}} \sum_i \left(N_i \tilde{d}_i^T \tilde{C}_i^{-1} \tilde{d}_i \right), \quad (13)$$

$$\tilde{d}_i = \frac{\sum_j b_j}{N_i} - \mathbf{T}a_i, \quad (14)$$

$$\tilde{C}_i = \frac{\sum_j C_j^B}{N_i} + \mathbf{T}C_i^A\mathbf{T}^T. \quad (15)$$

where N_i is the number of neighbor points. Eq. (13) suggests that we can efficiently compute the objective function by substituting the mean of the distributions of the points (b_j and C_j^B) around a_i for b_i and C_i^B in Eq. (5) and weighting the function by N_i .

We can naturally adapt this equation to voxel-based calculation by storing $b_i' = \frac{\sum b_j}{N_i}$ and $C_i' = \frac{\sum C_j^B}{N_i}$ in each voxel.

Fig. 1 illustrates the correspondence models used in GICP, the NDT, and our VGICP. GICP employs the nearest

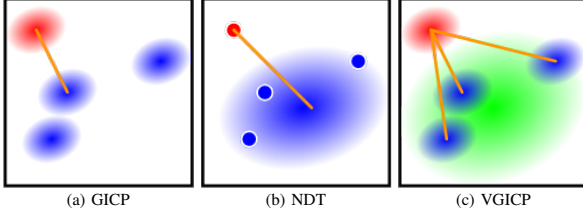


Fig. 1: Correspondence models for distance calculation in (a) GICP, (b) the NDT, and (c) VGICP. The red circles indicate a source point and the blue circles indicate target points. (GICP: nearest distribution-to-distribution, NDT: voxel-based point-to-distribution, VGICP: voxel-based distribution-to-multi-distribution.) The VGICP model yields a valid distribution even when a voxel contains only a few points.

distribution-to-distribution correspondence model, which is reasonable, but relies on costly nearest neighbor search. For fast registration, the NDT uses the point-to-voxel-distribution correspondence model. However, we need at least four points (more than ten in practice) to calculate a 3D covariance matrix. If the number of points in the voxel is low, the covariance matrix becomes corrupted. Our VGICP exploits single-to-multiple distributions in the voxel correspondences to deal with the case in which only a few points fall within a voxel. Because it calculates a voxel distribution from point distributions, it yields a proper covariance matrix even when the voxel contains only one point.

C. Implementation

Algorithm 1 describes the registration procedure of VGICP in detail. As stated above, the VGICP algorithm does not require costly nearest neighbor search during optimization, and it can hence leverage CPU and GPU parallel processing. For pose optimization, we choose the Gauss-Newton optimizer because it converges quickly and requires no hyper-parameters, unlike quasi-Newton methods.

We implemented three versions of the VGICP algorithm: single-threaded, multi-threaded, and GPU processing. All versions first estimate the covariance matrix of each point using a KD-tree-based nearest neighbor search [13]. This covariance estimation is parallelized in the multi-threaded and GPU processing versions. We also implemented a GPU-based brute force nearest neighbor search. As the baseline, we implemented the GICP algorithm in addition to VGICP. The GICP implementation is also parallelized by CPU multi-threading, is but not implemented on the GPU because it relies on KD-tree nearest neighbor search, which is not suitable for the GPU.

IV. EXPERIMENT

A. Simulated environment

We evaluated the proposed VGICP algorithm in a simulated environment. For this evaluation, we developed a realistic LIDAR data simulator. To generate realistic LIDAR

Algorithm 1 VGICP algorithm

```

1: Point clouds :  $\mathcal{A} = \{a_0, \dots, a_N\}$ ,  $\mathcal{B} = \{b_0, \dots, b_M\}$ 
2: Covariances :  $\mathcal{C}^A = \{C_0^A, \dots, C_N^A\}$ ,  $\mathcal{C}^B = \{C_0^B, \dots, C_M^B\}$ 
3: Initial guess:  $\mathbf{T}$ 
4: procedure VGICP( $\mathcal{A}, \mathcal{B}, \mathcal{C}^A, \mathcal{C}^B, \tilde{\mathbf{T}}$ )
5:    $\mathbf{T} \leftarrow \tilde{\mathbf{T}}$ 
6:    $\mathcal{V} \leftarrow \text{VOXELIZATION}(\mathcal{B}, \mathcal{C}^B)$ 
7:   while  $\mathbf{T}$  is not converged do
8:      $e = \emptyset, J = \emptyset$ 
9:     for  $i \in \{0, \dots, N\}$  do
10:      voxel_index  $\leftarrow \text{floor}(a_i/\text{voxel\_resolution})$ 
11:      if voxel_index  $\notin \mathcal{V}$  then  $\triangleright$  Not fall in a voxel
12:        continue
13:       $e_i, J_i \leftarrow \text{Cost}(\mathbf{T}, a_i, C_i^A, v, \mu, v, C, v, N)$ 
14:       $e \leftarrow e \cup e_i, J \leftarrow J \cup J_i$ 
15:       $\delta \mathbf{T} \leftarrow -(J^T J)^{-1} J^T e \triangleright$  Gauss-Newton update
16:       $\mathbf{T} \leftarrow \mathbf{T} \boxplus \delta \mathbf{T}$ 
17:   return  $\mathbf{T}$ 
18: procedure VOXELIZATION( $\mathcal{B}, \mathcal{C}^B$ )
19:    $\mathcal{V} \leftarrow \emptyset$ 
20:   for  $j \in \{0, \dots, M\}$  do
21:     voxel_index = cast_to_int( $b_j/\text{voxel\_resolution}$ )
22:     if voxel_index  $\notin \mathcal{V}$  then
23:        $\mathcal{V}[\text{voxel\_index}] \leftarrow (\mu = 0, C = 0, N = 0)$ 
24:        $\mathcal{V}[\text{voxel\_index}].\mu \leftarrow \mathcal{V}[\text{voxel\_index}].\mu + b_j$ 
25:        $\mathcal{V}[\text{voxel\_index}].C \leftarrow \mathcal{V}[\text{voxel\_index}].C + C_j^B$ 
26:        $\mathcal{V}[\text{voxel\_index}].N \leftarrow \mathcal{V}[\text{voxel\_index}].N + 1$ 
27:   for  $v \in \mathcal{V}$  do
28:      $v.\mu \leftarrow v.\mu/v.N$ 
29:      $v.C \leftarrow v.C/v.N$ 
30:   return  $\mathcal{V}$ 

```

data, the simulator first renders an omnidirectional depth image and then performs ray casting to generate Velodyne-like rotating LIDAR data. Fig. 2 shows examples of point clouds generated by our simulator and Microsoft AirSim [14]. Whereas our simulator generates realistic point clouds, Airsim and other popular simulators generate LIDAR data from collision models, and thus their point clouds are not realistic.

With this simulator, we generated a sequence of LIDAR point clouds ($\mathcal{P}_0, \dots, \mathcal{P}_N$) and corresponding sensor poses ($\hat{R}_0, \dots, \hat{R}_N$ and $\hat{p}_0, \dots, \hat{p}_N$). We used the parameters of Velodyne's HDL-32e sensor to simulate a real sensor model. We applied a scan matching method between consecutive frames and estimated the sensor pose (R_t, p_t) at time t by accumulating the scan matching results (i.e., the scan matching odometry). Following [15], we calculated the absolute trajectory error (ATE) and relative error (RE) to evaluate the accuracy of the scan matching methods.

The ATE is defined as follows:



Fig. 2: Examples of point clouds generated by our simulator and Microsoft AirSim. AirSim generates point clouds from collision models, and thus objects in the LIDAR data have shapes that are too simplified (see the points on the trees, cars, and buildings), and objects without collision models (pedestrians) do not appear in the point clouds. Our simulator performs raycasting on an omnidirectional depth image to generate realistic point clouds.

$$\text{ATE}_{\text{rot}} = \left(\frac{1}{N} \sum_i \|\angle(\Delta R_i)\|^2 \right)^{\frac{1}{2}}, \quad (16)$$

$$\text{ATE}_{\text{pos}} = \left(\frac{1}{N} \sum_i \|\Delta p_i\|^2 \right)^{\frac{1}{2}}, \quad (17)$$

where $\Delta R_i = R_i(\hat{R}_i)^{-1}$ and $\Delta p_i = p_i - \Delta R_i \hat{p}_i$. Note that the estimated sensor trajectories are aligned with respect to the ground truth using [16]. The RE is defined as follows:

$$\text{RE}_{\text{rot}} = \angle(\delta R_k) = \angle(R_e \hat{R}_e^T), \quad (18)$$

$$\text{RE}_{\text{pos}} = \|p_e - \delta R_k \hat{p}_e\|_2, \quad (19)$$

where $R_e = R_t^{-1} R_{t+N}$ and $p_e = p_t - p_{t+N}$. Parameters t and $t+N$ define the size of the window used to evaluate the RE. We set the window size to 1, 5, and 25 m (sensor travel distance) and evaluated the RE using a sliding window.

Fig. 3 shows the REs of different registration methods, and the absolute trajectory errors are shown in Table I. We evaluated our VGICP and GICP implementations as well as the ICP, GICP, and NDT implementations in Point Cloud Library (PCL). For VGICP and the NDT, we tested several voxel resolutions to evaluate the stability of their performance with respect to changes in hyper-parameter values.

The accuracy of the NDT heavily relies on the choice of voxel resolution. We can see that the registration accuracy of the NDT largely drops for voxel resolutions of 1, 2, and 6 m compared to the best one. In this evaluation, we observed the best accuracy with the NDT when the voxel resolution is 4.0 m. This value is, however, too large to capture the fine details of the environment, and thus the accuracy of the NDT is worse than that of GICP.

The GICP-based algorithms show superior accuracy compared to the classic ICP algorithm. Our VGICP and GICP implementations yield slightly better accuracy than does the PCL implementation of GICP. This might be due to the choice of the optimizer (our implementations use Gauss-Newton, which can be faster and more accurate than the

TABLE I: Absolute trajectory errors.

	[m]	[°]
GICP(ours)	0.562	1.696
VGICP(1.0m)	0.624	2.777
GICP(PCL)	0.954	1.724
ICP	75.349	32.171
NDT(4.0m)	7.952	23.971

Bold indicates the top three results.

Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm used in the PCL version of GICP). We note that our VGICP algorithm shows consistent results over a wide range of voxel resolutions thanks to the proposed voxelization approach, which yields a valid distribution even when the number of points in a voxel is low. This result illustrates that the proposed VGICP algorithm shows comparable accuracy with respect to GICP and is robust to hyper-parameter changes.

Fig. 4 shows the average processing times of the registration methods. All methods were run on an Intel Core i9-9900K and NVIDIA Geforce RTX2080Ti. For our GICP and VGICP implementations, we evaluated both the single and multi-threaded versions. We also evaluated the GPU version of our VGICP algorithm. The results show that our single-thread implementations (GICP: 189 msec, VGICP: 156 msec) are faster than the reference GICP implementation in PCL (201 msec). Because the preprocessing part (the estimation of the covariance matrix of each point) is the same for both GICP and VGICP, they spend the same amount of time for preprocessing. The following pose optimization part is greatly sped up by the voxelization approach. This suggests that in scenarios in which the target point cloud is not frequently changed (e.g., keyframe-based odometry estimation), we can obtain more benefits from the high efficiency of the VGICP algorithm. With multi-threading, our implementations of GICP and VGICP are further sped up, taking only 68 msec and 50 msec, respectively. Our GPU-implementation of VGICP is extremely fast and takes only 6 msec to optimize. It is worth mentioning that the GPU-

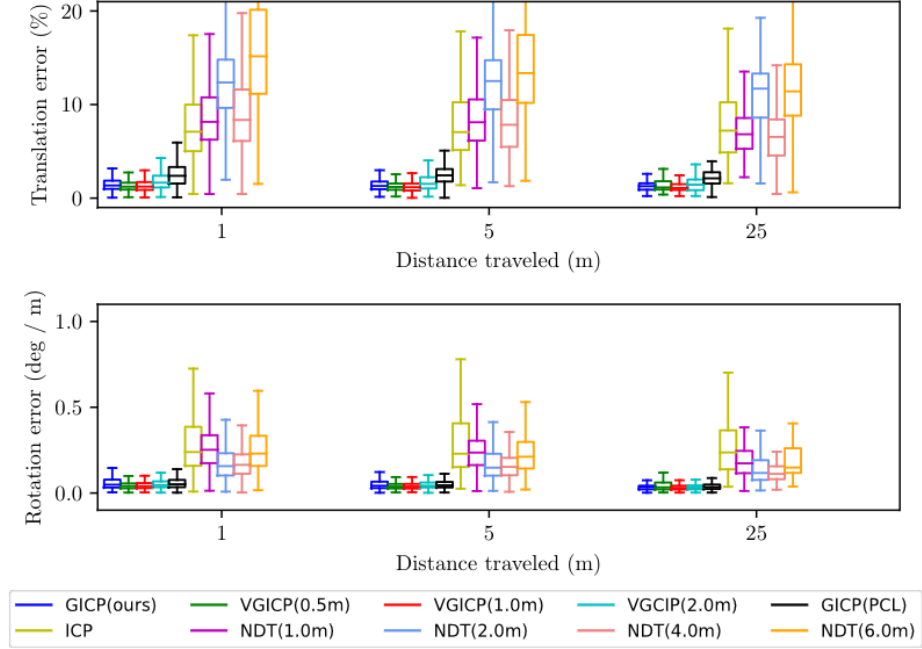


Fig. 3: REs of different registration methods.

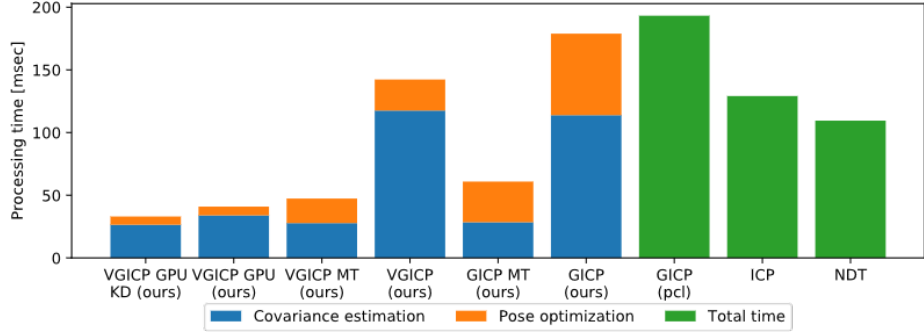


Fig. 4: Average processing time per scan.

based brute-force nearest neighbor search is slower than the CPU-based parallel KD-tree in this evaluation. However, it would be a reasonable choice for computers with a low-specification CPU and a high-performance GPU such as the NVIDIA Jetson.

B. Real environment

We recorded eight LIDAR sequences in the environment shown in Fig. 5 using a Velodyne HDL-32e sensor. The length of each trajectory is about 120m, and the number of points in each frame is about 15,000. Similar to the evaluation in Sec. IV-A, we estimated the sensor trajectory by applying registration algorithms between consecutive frames. For our implementations, we reused the calculated covariance matrices of an input point cloud in the registration of the next frame to reduce the preprocessing cost. For each

sequence, we aligned the last frame with respect to the first frame using GICP and used the result as ground truth.

Table II shows the pose estimation errors at the last frames (i.e., the accumulated registration errors). ICP and the NDT obtain worse results than the GICP-based algorithms. The NDT obtains the best accuracy (2.717m and 0.164° error) when the voxel resolution is set to 1.0 m. However, the accuracy drastically drops when the voxel resolution is too small or too large (7.119m and 0.443° for 0.5m resolution; 5.590m and 0.149° for 4.0m resolution).

The PCL-based GICP obtains a better result than do ICP and the NDT (1.316m and 0.051°). However, because it heavily relies on costly nearest neighbor search, it is the slowest of the evaluated methods (5.2 fps).

Our GICP implementation is substantially sped up by multi-threading and the fast Gauss-Newton optimizer (20.5



Fig. 5: Experimental environment.

TABLE II: Processing speed and transformation errors

Method	FPS (CPU/GPU)	Translation[m]	Rotation[°]
VGICP (0.5m)	28.7/116.5	0.852 $\pm$ 0.289	0.049 $\pm$ 0.013
VGICP (1.0m)	30.4/120.1	1.177 $\pm$ 0.456	0.048 $\pm$ 0.026
GICP (ours)	20.5	0.893 $\pm$ 0.210	0.045 $\pm$ 0.023
GICP (PCL)	5.2	1.316 $\pm$ 0.310	0.051 $\pm$ 0.022
NDT (0.5m)	8.3	7.119 $\pm$ 5.640	0.443 $\pm$ 0.331
NDT (1.0m)	10.3	2.717 $\pm$ 2.645	0.164 $\pm$ 0.088
NDT (2.0m)	10.4	3.290 $\pm$ 1.420	0.116 $\pm$ 0.066
NDT (4.0m)	9.3	5.590 $\pm$ 2.177	0.149 $\pm$ 0.077
ICP	11.0	10.192 $\pm$ 4.365	0.371 $\pm$ 0.106

Bold indicates the top three results.

fps). It also obtains a higher accuracy than the PCL-based GICP (0.893m and 0.045°). The VGICP algorithm has almost the same accuracy as our GICP implementation (0.852m and 0.049°) when the voxel resolution is set to 0.5 m. Although the accuracy of VGICP slightly drops (1.177m and 0.048°) when the voxel resolution is changed to 1.0 m, the result is still comparable to that of GICP. Our VGICP algorithm leverages CPU multi-threading, which results in the best processing speed (about 30 fps). Furthermore, it becomes extremely fast on the GPU (about 120 fps).

V. CONCLUSION AND FUTURE WORK

In this study, we proposed the voxelized GICP algorithm. The proposed VGICP is as accurate as GICP because it utilizes the voxel-based association approach. The evaluation results in the simulated and real environments show that the proposed method shows a superior processing speed (30fps on a CPU and 120 fps on a GPU) and is robust to voxel resolution changes.

We plan to evaluate and improve the convergence of the propose VGICP algorithm because it employs the voxelization approach which may affect the registration result when the initial guess is not close to the true pose. We also plan

to develop an IMU-LIDAR joint optimization-based localization method with VGICP, which requires an extremely fast evaluation of the relative poses between keyframes.

ACKNOWLEDGEMENT

This work was partly supported by a project commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

REFERENCES

- [1] A. Segal, D. Haehnel, and S. Thrun, “Generalized-ICP,” in *Robotics: Science and Systems*. Robotics: Science and Systems Foundation, jun 2009.
- [2] P. Biber and W. Strasser, “The normal distributions transform: a new approach to laser scan matching,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2003.
- [3] M. Magnusson, A. Nuchter, C. Lorken, A. Lilienthal, and J. Hertzberg, “Evaluation of 3d registration reliability and speed - a comparison of ICP and NDT,” in *IEEE International Conference on Robotics and Automation*. IEEE, may 2009.
- [4] J. Zhang and S. Singh, “LOAM: Lidar odometry and mapping in real-time,” in *Robotics: Science and Systems*. Robotics: Science and Systems Foundation, jul 2014.
- [5] D. Chetverikov, D. Svirko, D. Stepanov, and P. Krsek, “The trimmed iterative closest point algorithm,” in *Object recognition supported by user interaction for service robots*. IEEE, Aug. 2002.
- [6] J. Serafin and G. Grisetti, “NICP: Dense normal based point cloud registration,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, sep 2015.
- [7] T. Stoyanov, M. Magnusson, H. Andreasson, and A. J. Lilienthal, “Fast and accurate scan registration through minimization of the distance between compact 3d NDT representations,” *International Journal of Robotics Research*, vol. 31, no. 12, pp. 1377–1393, sep 2012.
- [8] M. Magnusson, N. Vaskevicius, T. Stoyanov, K. Pathak, and A. Birk, “Beyond points: Evaluating recent 3d scan-matching algorithms,” in *IEEE International Conference on Robotics and Automation*. IEEE, may 2015.
- [9] E. Takeuchi and T. Tsubouchi, “A 3-d scan matching using improved 3-d normal distributions transform for mobile robotic mapping,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, oct 2006.
- [10] T. Shan and B. Englot, “LeGO-LOAM: Lightweight and ground-optimized lidar odometry and mapping on variable terrain,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, oct 2018.
- [11] R. B. Rusu, N. Blodow, and M. Beetz, “Fast point feature histograms (FPFH) for 3d registration,” in *IEEE International Conference on Robotics and Automation*. IEEE, may 2009.
- [12] S. Salti, F. Tombari, and L. D. Stefano, “SHOT: Unique signatures of histograms for surface and texture description,” vol. 125. Elsevier, aug 2014, pp. 251–264.
- [13] J. L. Bentley, “Multidimensional binary search trees used for associative searching,” vol. 18, no. 9. ACM, sep 1975, pp. 509–517.
- [14] S. Shah, D. Dey, C. Lovett, and A. Kapoor, “AirSim: High-fidelity visual and physical simulation for autonomous vehicles,” in *Field and Service Robotics*. Springer, nov 2017, pp. 621–635.
- [15] Z. Zhang and D. Scaramuzza, “A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, oct 2018.
- [16] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” vol. 13, no. 4. IEEE, apr 1991, pp. 376–380.